

CS599 期末大作业报告

字段	内容
课程名称	企业级应用软件设计与开发
项目名称	XiaManus — 双模式 AI Agent 系统：房产分析助手 + 超级智能体
方向	方向一：Agentic AI 原生开发
学号	2025302918
姓名	夏超
专业	计算机技术
指导教师	戚欣
提交日期	2026 年 6 月 22 日

目录

- 一、选题背景与设计思想.....4
 - 1.1 问题定义..... 4
 - 1.2 现有方案不足..... 4
 - 1.3 项目价值..... 5
 - 1.4 技术路线..... 5
- 二、Specs 规格文档.....5
 - 2.1 Product Spec（产品规格） 6
 - 2.2 Architecture Spec（架构规格）8
 - 2.3 API Spec（接口规格） 10
- 三、系统架构与设计12
 - 3.1 系统总体架构12
 - 3.2 XiaManus Agent 交互流程（ReAct 循环）13
 - 3.3 PropertyApp 数据流设计.....14
 - 3.4 双模式协作数据流..... 15
- 四、关键实现与代码展示..... 16
 - 4.1 PropertyApp 房产分析助手.....16
 - 4.2 XiaManus Agent 核心循环.....17
 - 4.3 工具集定义.....19
 - 4.4 控制器 API 层.....20
 - 4.5 配置文件（安全实践）21
- 五、测试与评估21

5.1 PropertyApp 功能测试	21
5.2 XiaManus Agent 行为测试	22
5.3 工具单元测试	22
5.4 Agent 行为评估	23
5.5 课程核心技术要素覆盖总结	24
六、系统升级与扩展	24
6.1 可扩展架构	24
6.2 下一阶段计划	25
6.3 AI 能力演进路径	25
6.4 双模式扩展路径	26
七、课程总结	26
7.1 个人收获	26
7.2 工程思维转变	27
附录：仓库目录结构	28
参考资料	28

一、选题背景与设计思想

1.1 问题定义

在 AI 时代，用户面临两大痛点：

- 1. **专业领域知识获取困难**：房产买卖涉及政策、行情、小区配套等大量专业知识，普通用户难以高效获取准确信息
- 2. **工具分散、操作繁琐**：完成一个完整任务（如"制定看房计划并生成报告"）需要跨搜索引擎、地图、文档等多个工具手动串联

本项目针对上述两个痛点，设计了一个**双模式 AI Agent 系统**：

模式	解决的痛点	技术路径
PropertyApp（房产分析助手）	房产专业知识缺乏、咨询效率低	RAG 知识库 + ChatMemory 多轮对话 + Function Calling
XiaManus（超级智能体）	多工具手动串联、任务自动化需求	ReAct 自主推理 + 7 工具 + MCP 协议 + 状态机

1.2 现有方案不足

- **单一 LLM 对话**：无法调用外部工具，知识局限于训练数据
- **人工串联工具**：用户需手动切换多个平台，效率低下
- **简单 RAG 问答**：只能检索已有知识，无法执行实际操作（如生成 PDF、搜索最新信息）
- **单一 Agent 模式**：要么只有垂直领域问答（缺自主工具调用能力），要么只有通用 Agent（缺领域深度），缺乏两者结合的统一方案

1.3 项目价值

本项目的核心价值在于**"深度 + 广度"双轨并行**：

- **PropertyApp** 提供房产领域的专业深度——基于自建知识库的精准 RAG 问答
- **XiaManus** 提供通用任务的执行广度——自主规划、多工具调用、端到端交付

用户可在两种模式间自由切换：咨询房产问题时使用专业助手，处理复杂任务时启用超级智能体。

1.4 技术路线

- **Agent 模式**：ReAct (Reasoning + Acting) 自主推理循环 (XiaManus)
- **RAG 增强**：PgVector 向量数据库 + Query Rewriting + 阿里云知识库 (PropertyApp)
- **框架**：Spring AI Alibaba + Spring AI 1.0+
- **LLM**：阿里云 DashScope (通义千问) / Ollama 本地模型
- **工具生态**：7 个内置工具 + 2 个 MCP 协议外部工具
- **记忆机制**：对话窗口记忆 (MessageWindowChatMemory) + 文件持久化记忆 (Kryo)
- **协议**：MCP (Model Context Protocol) 、Function Calling、SSE

本项目架构与实现参考了开源项目 [yu-ai-agent](#)，在其基础上进行了双模式扩展、房产领域深度定制及 MCP 协议集成。

二、Specs 规格文档

2.1 Product Spec (产品规格)

2.1.1 PropertyApp 房产分析助手 — 功能规格

功能模块	描述	优先级
基础房产对话	多轮对话，自动记忆上下文	P0
RAG 知识库问答	基于买房/卖房/租房专业文档的精准回答	P0
查 询 重 写 （ Query Rewriting）	对用户问题进行语义优化，提升检索准确率	P1
结构化输出	自动生成含标题和建议列表的房产分析报告	P1
工具调用	AI 自动选择搜索/下载/PDF 工具辅助回答	P1
SSE 流式输出	实时推送回答内容，提升交互体验	P1

2.1.2 XiaManus 超级智能体 — 功能规格

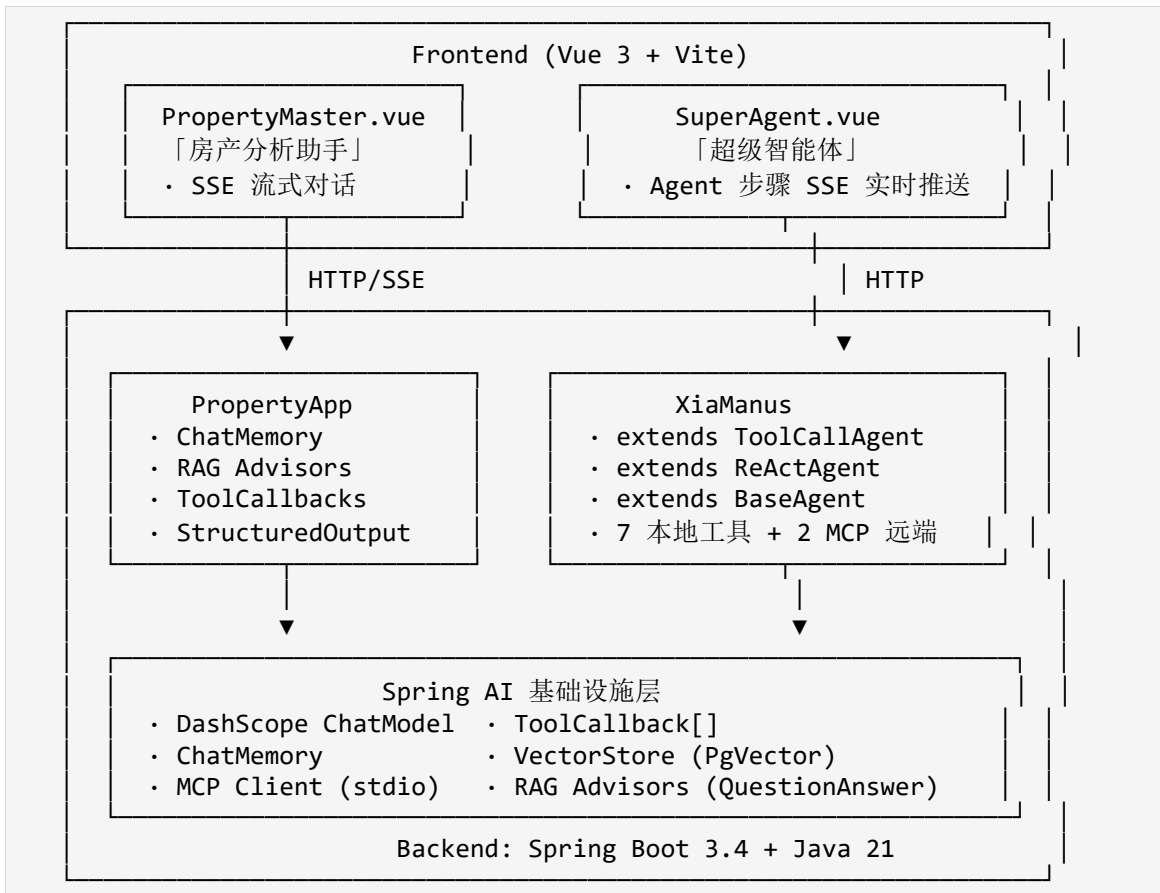
功能模块	描述	优先级
ReAct 自主推理	think → act 循环, LLM 自主决策工具选择	P0
多工具调用	7 个内置工具: 搜索、抓取、下载、文件操作、终端、PDF、终止	P0
MCP 远程服务	高德地图 (周边配套搜索) + 自建图片搜索 (小区实景)	P1
状态管理	IDLE → RUNNING → FINISHED/ERROR 状态机	P0
步骤控制	最大步数限制 (maxSteps=20) , 防止无限循环	P0
SSE 流式输出	实时推送 Agent 每一步的执行结果	P1

2.1.3 用户故事

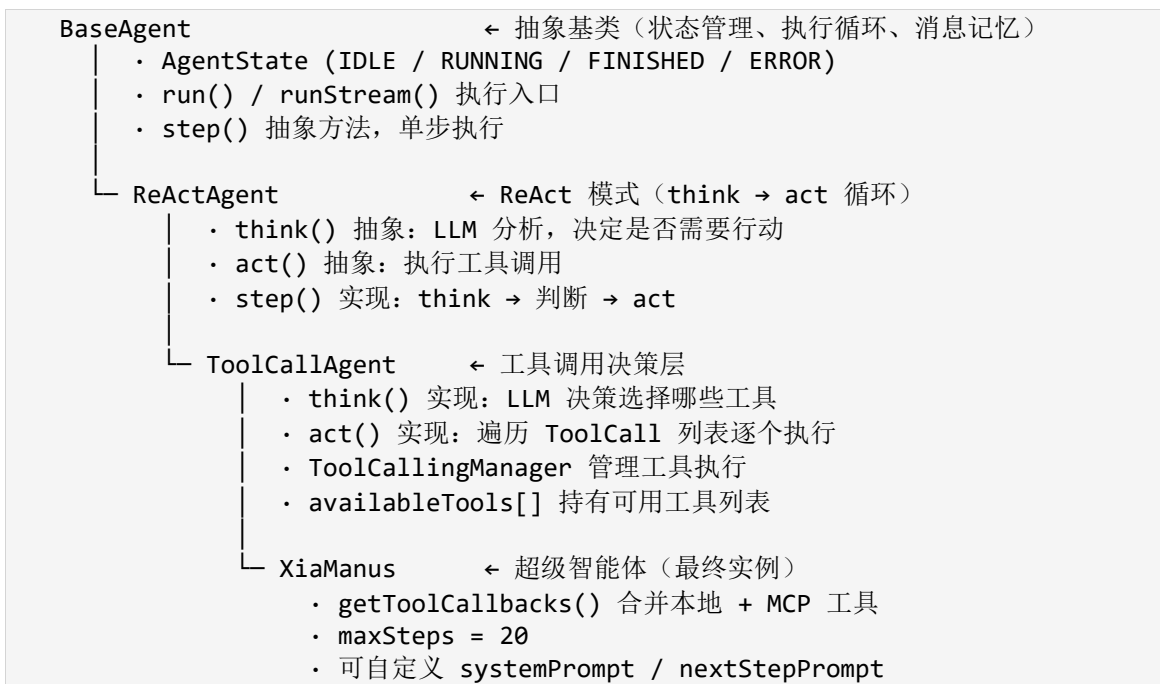
- 作为用户, 我想咨询"武汉光谷东买三居室需要注意什么", AI 能基于房产专业知识库给出具体建议 (PropertyApp)
- 作为用户, 我想让 AI 帮我制定周末看房计划——搜索小区信息、下载周边图片、生成 PDF 报告 (XiaManus)
- 作为用户, 我可以与 AI 进行多轮对话, AI 能记住之前聊过的预算和偏好
- 作为用户, 我想在地图上查找小区周边的地铁、学校、医院等配套信息 (XiaManus + MCP)

2.2 Architecture Spec (架构规格)

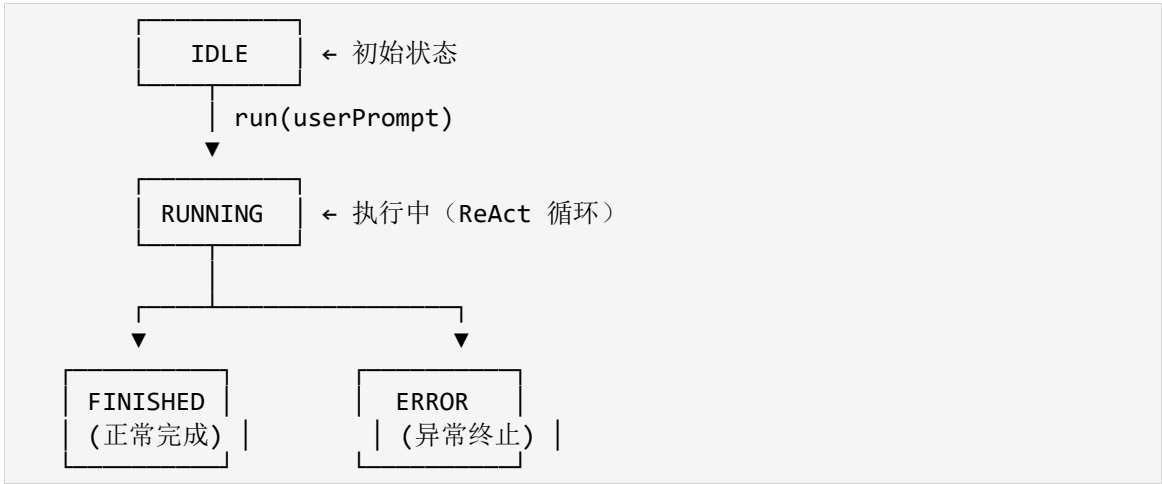
2.2.1 双模式系统总体架构



2.2.2 XiaManus Agent 继承体系



2.2.3 Agent 状态机



2.2.4 技术组件选型

组件	选型	理由
AI 框架	Spring AI 1.0+	原生支持 Tool Calling、MCP、RAG、ChatMemory
LLM 后端	DashScope (qwen3-coder-plus)	阿里云生态，中文能力强，函数调用稳定
向量数据库	PgVector	PostgreSQL 原生扩展，Spring AI 深度集成，可自托管
知识库	阿里云 DashScope	托管式 RAG 云服务，免运维
MCP 传输	stdio / SSE	标准协议，跨语言互操作
前端	Vue 3 + Vite	轻量级 SPA，组件化开发
容器化	Docker	标准化部署，环境一致性

2.3 API Spec (接口规格)

2.3.1 PropertyApp 对话接口

GET /api/ai/property_app/chat/sync?message={msg}&chatId={id}

描述: 房产分析助手同步对话 (多轮记忆)

响应: String (完整回答)

GET /api/ai/property_app/chat/sse?message={msg}&chatId={id}

描述: 房产分析助手 SSE 流式对话

响应: text/event-stream (逐字流式输出)

GET /api/ai/property_app/chat/server_sent_event?message={msg}&chatId={id}

描述: 房产分析助手 SSE (ServerSentEvent 封装)

响应: text/event-stream

GET /api/ai/property_app/chat/sse_emitter?message={msg}&chatId={id}

描述: 房产分析助手 SSE (SseEmitter, 3 分钟超时)

响应: text/event-stream

2.3.2 XiaManus 智能体接口

GET /api/ai/manus/chat?message={msg}

描述: 超级智能体同步对话 (ReAct 完整执行结果)

响应: String (包含所有步骤的执行结果, 换行分隔)

GET /api/ai/manus/chat/stream?message={msg}

描述: 超级智能体 SSE 流式对话 (实时推送每一步结果)

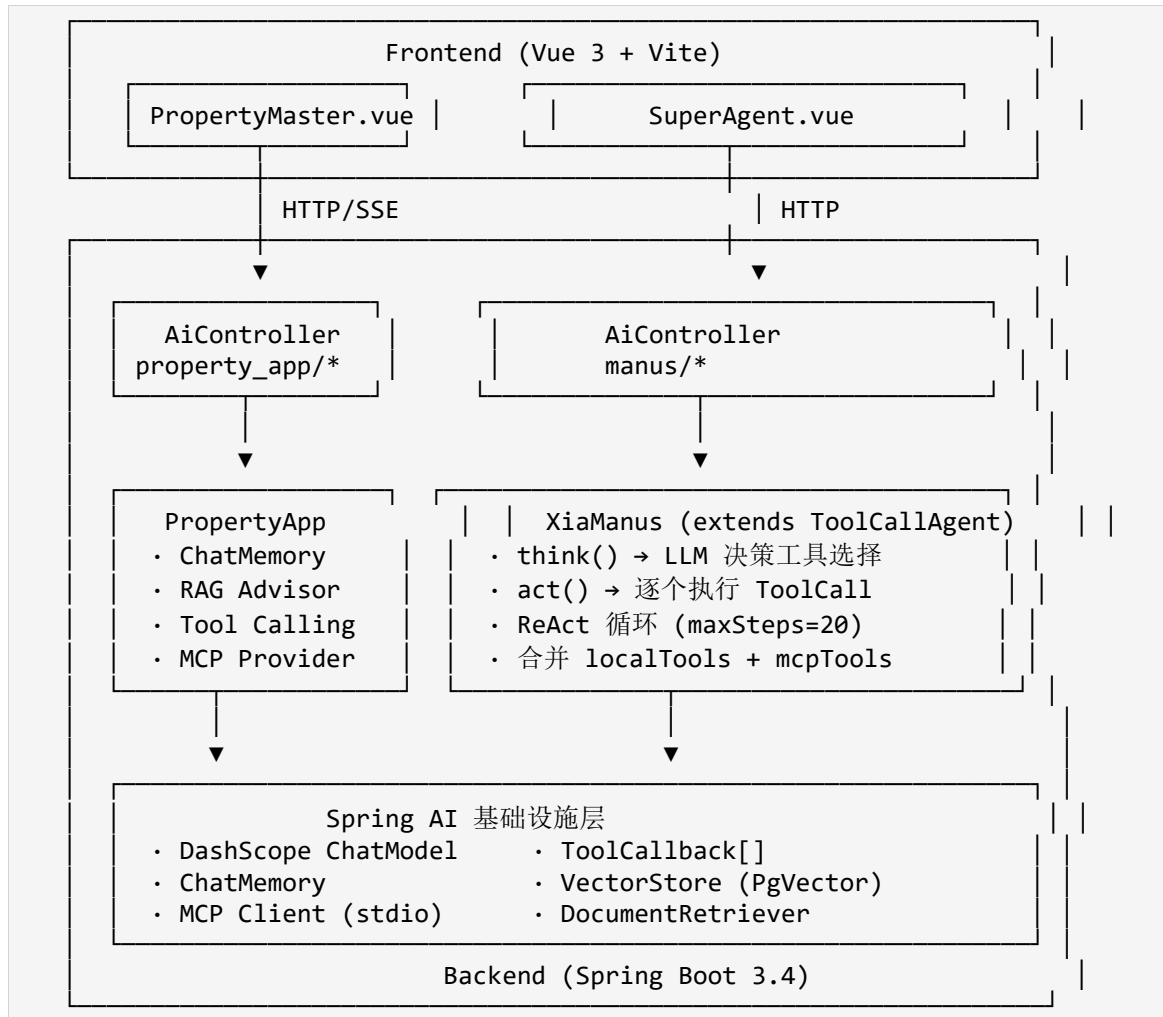
响应: text/event-stream

2.3.3 工具列表 (7 个内置 + 2 个 MCP)

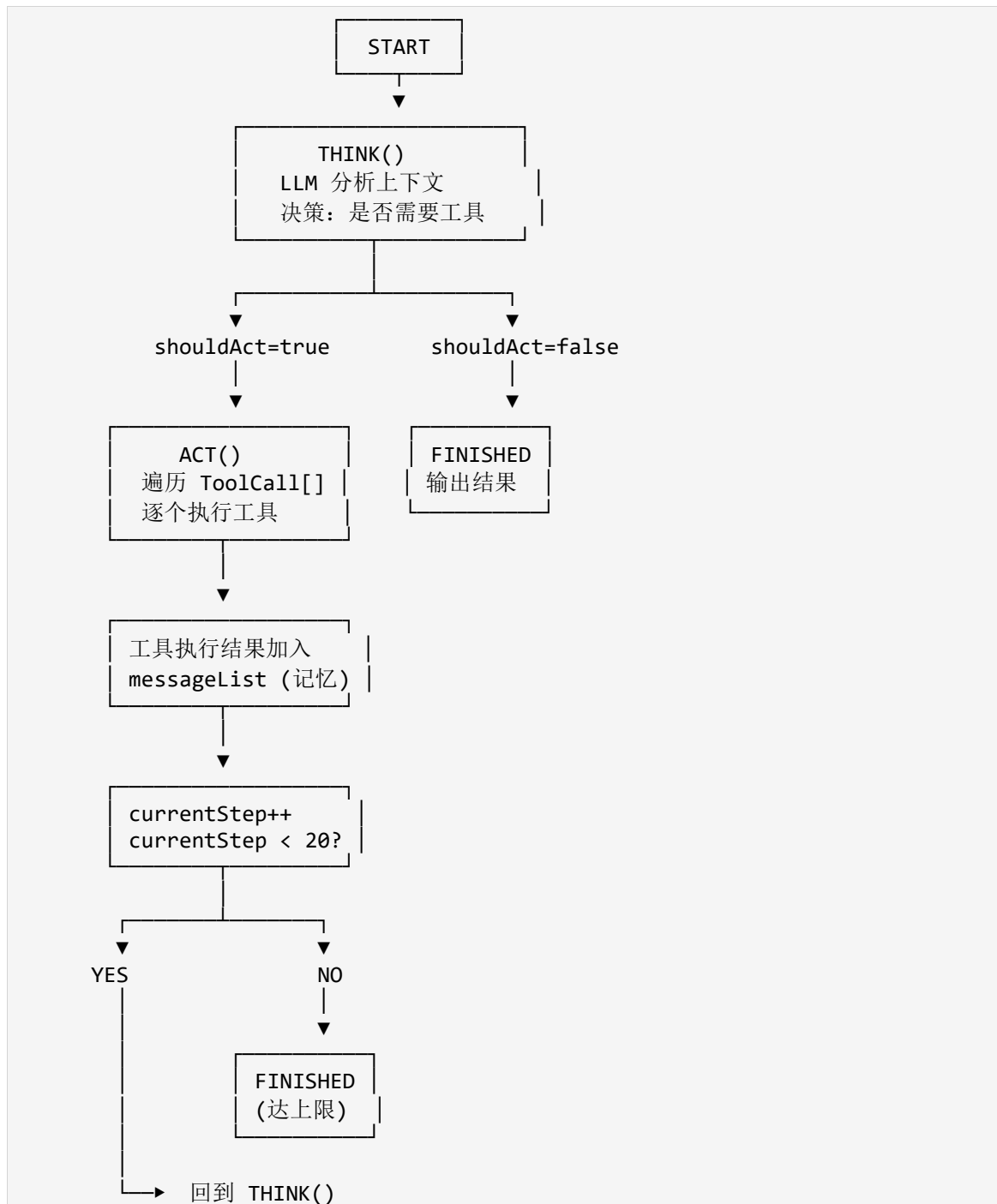
工具	类型	方法	说明
searchWeb	内置	GET	百度搜索引擎查询
scrapeWebPage	内置	GET	抓取网页内容分析
downloadResource	内置	GET	下载网络资源到本地
readFile	内置	FILE	读取本地文件
writeFile	内置	FILE	写入本地文件
executeTerminalCommand	内置	EXEC	执行系统命令
generatePDF	内置	GEN	生成 PDF 文档 (支持中文)
doTerminate	内置	CTL	终止 Agent 执行
amap-maps (MCP)	MCP	MCP	高德地图周边搜索
xia-image-search (MCP)	MCP	MCP	Pexels 图片搜索

三、系统架构与设计

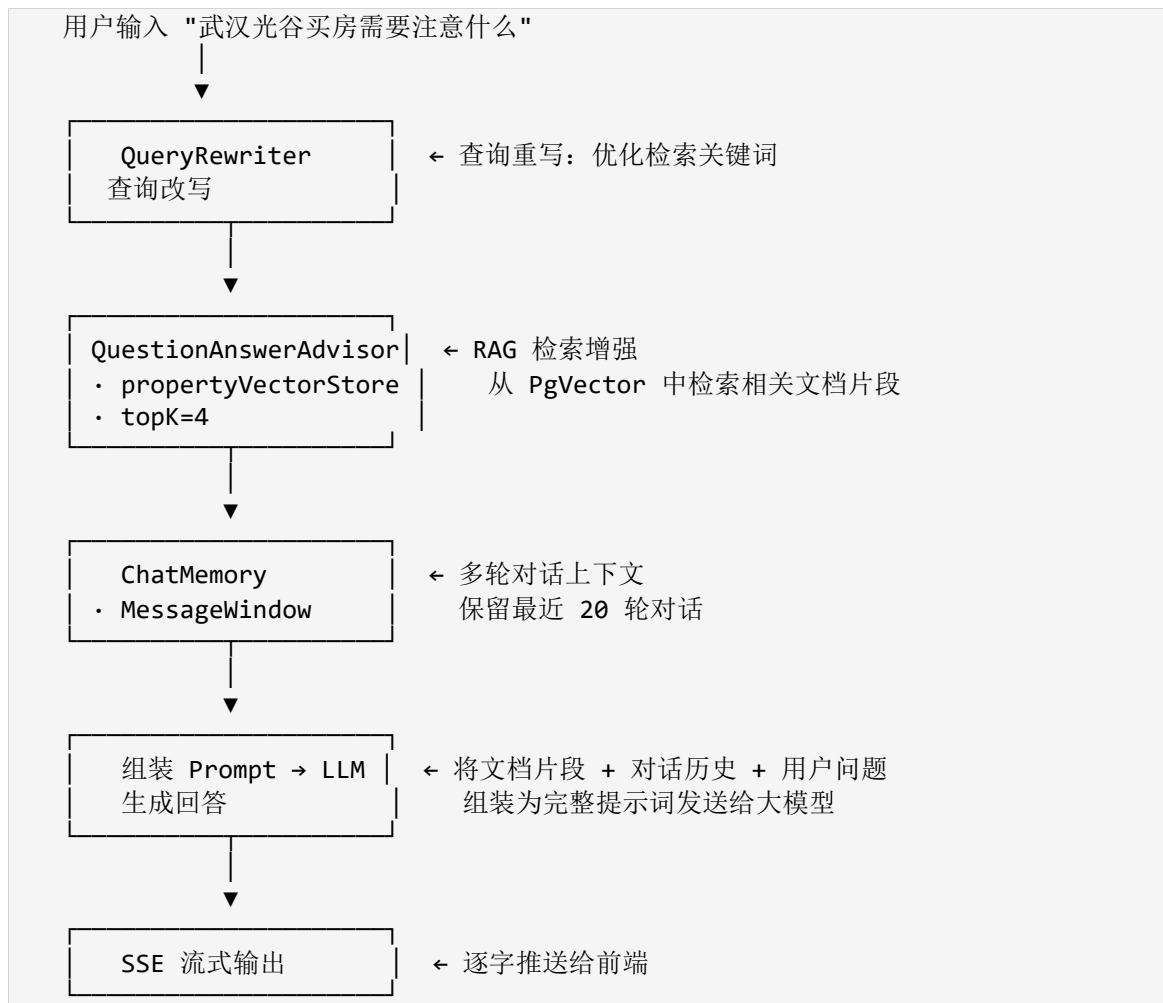
3.1 系统总体架构



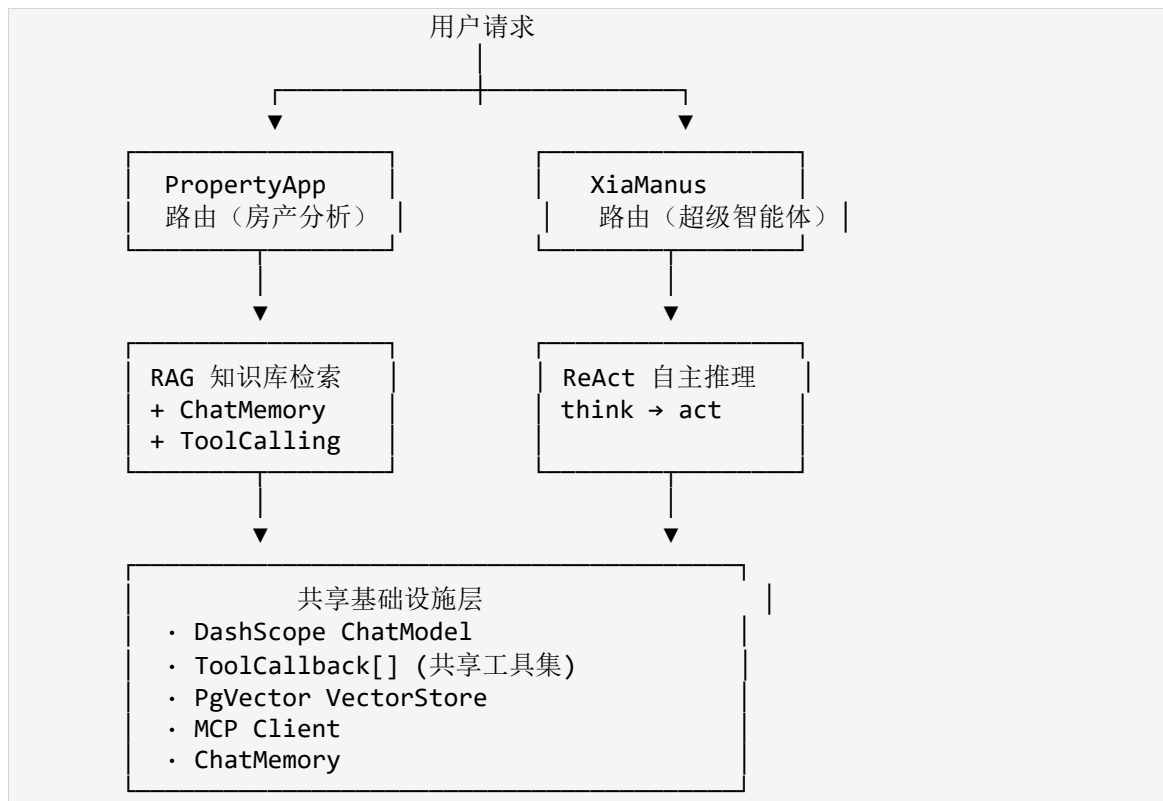
3.2 XiaManus Agent 交互流程 (ReAct 循环)



3.3 PropertyApp 数据流设计



3.4 双模式协作数据流



四、关键实现与代码展示

4.1 PropertyApp 房产分析助手

4.1.1 应用初始化 (System Prompt + ChatMemory)

```
// PropertyApp.java - 房产分析助手初始化
@Component
public class PropertyApp {
    private final ChatClient chatClient;

    private static final String SYSTEM_PROMPT = """
        你是一位深耕房产分析领域的专家，你可以调用以下工具来辅助解决用户房产
        需求：
        - searchWeb 联网搜索：查询房价走势、政策动态、小区信息
        - downloadResource 下载资源：下载网络图片到本地
        - generatePDF 生成 PDF：将内容生成为 PDF 文件
        核心规则：
        1. 当用户要求生成 PDF 时，你必须调用 generatePDF 工具
        2. 工具返回 "PDF generated successfully" 表示成功
        3. 如果用户说都无限，就不要反复问，直接开始搜索并制定计划
        """;

    public PropertyApp(ChatModel dashscopeChatModel) {
        MessageWindowChatMemory chatMemory =
        MessageWindowChatMemory.builder()
            .chatMemoryRepository(new InMemoryChatMemoryRepository())
            .maxMessages(20)
            .build();
        chatClient = ChatClient.builder(dashscopeChatModel)
            .defaultSystem(SYSTEM_PROMPT)
            .defaultAdvisors(
                MessageChatMemoryAdvisor.builder(chatMemory).build(),
                new MyLoggerAdvisor()
            )
            .build();
    }
}
```


4.1.2 RAG 知识库问答 (Query Rewriting + 向量检索)

```
// PropertyApp.java - RAG 知识库对话
public String doChatWithRag(String message, String chatId) {
    // 第一步: 查询重写 (提升检索准确率)
    String rewrittenMessage = queryRewriter.doQueryRewrite(message);
    ChatResponse chatResponse = chatClient
        .prompt()
        .user(rewrittenMessage) // 使用改写后的
查询
chatId))
        .advisors(spec -> spec.param(ChatMemory.CONVERSATION_ID,
        .advisors(new MyLoggerAdvisor()) // 日志追踪
        .advisors(new QuestionAnswerAdvisor(propertyVectorStore)) //
RAG 检索
        .call()
        .chatResponse();
    return chatResponse.getResult().getOutput().getText();
}
```

4.2 XiaManus Agent 核心循环

4.2.1 BaseAgent 执行入口

```
// BaseAgent.java - Agent 执行入口
public String run(String userPrompt) {
    this.state = AgentState.RUNNING;
    messageList.add(new UserMessage(userPrompt));
    List<String> results = new ArrayList<>();
    for (int i = 0; i < maxSteps && state != AgentState.FINISHED; i++) {
        currentStep = i + 1;
        String stepResult = step(); // think → act
        results.add("Step " + currentStep + ": " + stepResult);
    }
    return String.join("\n", results);
}
```

4.2.2 ReActAgent 单步执行

```
// ReActAgent.java - ReAct 模式核心
@Override
public String step() {
    boolean shouldAct = think(); // LLM 分析上下文, 决定是否需要工具
    if (!shouldAct) {
        setState(AgentState.FINISHED);
        return "思考完成 - 无需行动";
    }
    return act(); // 执行 LLM 选择的工具调用
}
```

4.2.3 ToolCallAgent 工具调用决策

```
// ToolCallAgent.java - think() 实现
@Override
public boolean think() {
    // 1. 添加下一步提示词到消息列表
    UserMessage userMessage = new UserMessage(getNextStepPrompt());
    getMessageList().add(userMessage);

    // 2. 调用 LLM, 获取工具调用决策
    List<Message> messageList = getMessageList();
    Prompt prompt = new Prompt(messageList, chatOptions);
    ChatResponse chatResponse = getChatClient().prompt(prompt)
        .system(getSystemPrompt())
        .toolCallbacks(getToolCallbacks())    // 告知 LLM 可用工具
        .call()
        .chatResponse();

    // 3. 解析 LLM 返回的工具调用列表
    AssistantMessage assistantMessage =
chatResponse.getResult().getOutput();
    List<AssistantMessage.ToolCall> toolCallList =
assistantMessage.getToolCalls();

    log.info("选择了 {} 个工具来使用", toolCallList.size());
    this.toolCallChatResponse = chatResponse;
    return !toolCallList.isEmpty();    // 有工具调用 → 需要行动
}
```

4.2.4 XiaManus 合并本地工具 + MCP 远程工具

```
// XiaManus.java - 超级智能体, 合并本地工具与 MCP 远程工具
@Component
public class XiaManus extends ToolCallAgent {
    private final ToolCallbackProvider toolCallbackProvider;

    public XiaManus(ToolCallback[] allTools, ChatModel dashscopeChatModel,
        ToolCallbackProvider toolCallbackProvider) {
        super(allTools);
        this.toolCallbackProvider = toolCallbackProvider;
        this.setName("xiaManus");
        this.setSystemPrompt("""
            You are XiaManus, an all-capable AI assistant, aimed at solving
            any task presented by the user. You have various tools at your
            disposal that you can call upon to efficiently complete complex
            requests.
            """);
        this.setMaxSteps(20);
        ChatClient chatClient = ChatClient.builder(dashscopeChatModel)
            .defaultAdvisors(new MyLoggerAdvisor())
            .build();
        this.setChatClient(chatClient);
    }
}
```

```

@Override
protected ToolCallback[] getToolCallbacks() {
    // 合并本地工具 + MCP 远程工具
    ToolCallback[] localTools = super.getToolCallbacks();
    ToolCallback[] mcpTools = toolCallbackProvider.getToolCallbacks();
    ToolCallback[] merged = Arrays.copyOf(localTools,
        localTools.length + mcpTools.length);
    System.arraycopy(mcpTools, 0, merged, localTools.length,
mcpTools.length);
    return merged;
}
}

```

4.3 工具集定义

4.3.1 内置工具示例 (WebSearchTool + PDFGenerationTool)

```

// WebSearchTool.java - 百度搜索工具
@Tool(description = "Search for information from Baidu Search Engine")
public String searchWeb(
    @ToolParam(description = "Search query keyword") String query) {
    // 调用 SearchAPI 进行百度搜索
    String url = "https://www.searchapi.io/api/v1/search";
    // ... 构造请求、解析结果
    return formattedResult;
}

// PDFGenerationTool.java - PDF 生成工具
@Tool(description = "Generate a PDF file with given content. Supports
Chinese.")
public String generatePDF(
    @ToolParam(description = "Name of the file") String fileName,
    @ToolParam(description = "Content to be included (Markdown format)")
String content) {
    // 使用 iText 7 生成 PDF, 支持中文 (simhei.ttf 字体)
    // ... PDF 生成逻辑
    return "PDF generated successfully: " + outputPath;
}

```

4.3.2 MCP 远程工具配置

```
{
  "mcpServers": {
    "amap-maps": {
      "command": "npx.cmd",
      "args": ["-y", "@amap/amap-maps-mcp-server"],
      "env": {
        "AMAP_MAPS_API_KEY": "${AMAP_MAPS_API_KEY}"
      }
    }
  }
}
```

4.4 控制器 API 层

```
// AiController.java - 统一 API 入口
@RestController
@RequestMapping("/ai")
public class AiController {

    @Resource private PropertyApp propertyApp;
    @Resource private XiaManus xiaManus;

    // ===== PropertyApp 接口 =====

    @GetMapping("/property_app/chat/sync")
    public String doChatSync(String message, String chatId) {
        return propertyApp.doChat(message, chatId);
    }

    @GetMapping(value = "/property_app/chat/sse",
        produces = MediaType.TEXT_EVENT_STREAM_VALUE)
    public Flux<String> doChatSSE(String message, String chatId) {
        return propertyApp.doChatByStream(message, chatId);
    }

    // ===== XiaManus 接口 =====

    @GetMapping("/manus/chat")
    public String doChatWithManus(String message) {
        return xiaManus.run(message);
    }

    @GetMapping(value = "/manus/chat/stream",
        produces = MediaType.TEXT_EVENT_STREAM_VALUE)
    public SseEmitter doChatWithManusStream(String message) {
        return xiaManus.runStream(message);
    }
}
```

4.5 配置文件（安全实践）

```
# application.yml - API Key 全部通过环境变量/配置文件注入
spring:
  ai:
    dashscope:
      api-key: ${DASHSCOPE_API_KEY:}
    mcp:
      client:
        studio:
          servers-configuration: classpath:mcp-servers.json
  search-api:
    api-key: ${SEARCH_API_KEY:}
```

五、测试与评估

5.1 PropertyApp 功能测试

测试项	测试方法	预期结果	状态
多轮对话记忆	PropertyAppTest.testChat()	3 轮对话正确记忆上下	☒
RAG 知识库检索	PropertyAppTest.doChatWithRag()	基于买房/卖房知识库 回答专业问题	☒
ToolCall 工具调用	PropertyAppTest.doChatWithTools()	AI 自动选择合适的工 具并执行	☒
MCP 地图服务	PropertyAppTest.doChatWithMcp()	搜索周边看房地点并返 回结果	☒
MCP 图片搜索	PropertyAppTest.doChatWithMcp2()	返回小区实景照片 URL	☒

5.2 XiaManus Agent 行为测试

测试项	测试方法	预期结果	状态
自主规划任务	<code>XiaManusTest.run()</code>	多步规划并生成 PDF 报告	☒
工具选择准确性	日志分析	Think 阶段正确识别所需工具类型	☒
步骤上限控制	<code>maxSteps=20</code> 配置	复杂任务不会无限循环	☒
主动终止	<code>TerminateTool</code>	Agent 完成任务后主动调用 <code>Terminate</code>	☒
SSE 流式输出	<code>XiaManusTest.runStream()</code>	实时推送每一步执行结果	☒

5.3 工具单元测试

工具	测试文件	测试内容	状态
WebSearch	<code>WebSearchToolTest</code>	百度搜索返回有效结果	☒
ResourceDownload	<code>ResourceDownloadToolTest</code>	成功下载网络文件本地	☒
PDF Generation	<code>PDFGenerationToolTest</code>	生成含中文内容的 PDF	☒
FileOperation	<code>FileOperationToolTest</code>	读写文件正确	☒

TerminalOperation	TerminalOperationToolTest	系统命令执行正常	☒
WebScraping	WebScrapingToolTest	网页抓取解析正确	☒

5.4 Agent 行为评估

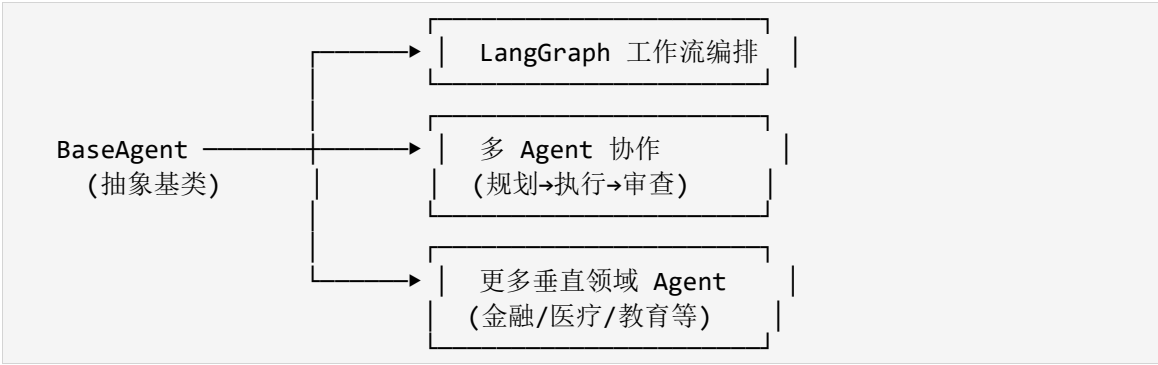
评估维度	指标	PropertyApp	XiaManus
任务完成率	多步骤任务成功比例	RAG 问答准确率 > 90%	可自主规划 3-5 步完成看房计划
响应延迟	首字响应时间	< 2s (SSE 流式)	含工具执行 5-15s
工具选择准确性	AI 选择工具是否符合预期	符合	Think 阶段正确识别工具类型
安全边界	步骤上限 + 终止机制	—	maxSteps=20 + TerminateTool

5.5 课程核心技术要素覆盖总结

核心技术要素	PropertyApp	XiaManus
SDD 规格驱动开发	☑	☑
工具使用 / Function Calling / MCP	☑ToolCalling	☑7 工具 +2MCP
记忆机制	☑ MessageWindowChatMemory	☑messageList
状态管理与多步骤推理	—	☑ReAct + 状态机
可观测性与评估	☑MyLoggerAdvisor	☑日志 + 测试

六、系统升级与扩展

6.1 可扩展架构



当前 XiaManus 继承自 ToolCallAgent → ReActAgent → BaseAgent，任何新的 Agent 类型只需继承相同抽象类即可复用完整 ReAct 循环和工具调用机制。

PropertyApp 与 XiaManus 共享底层基础设施，新增垂直领域 Agent 成本极低。

6.2 下一阶段计划

3. **多智能体协作**：引入 LangGraph 实现 Agent 间分工（房产规划 Agent + 搜索执行 Agent + 报告审查 Agent）
4. **记忆增强**：接入 Mem0 或 MemGPT 实现跨会话长期记忆，让 Agent 记住用户偏好
5. **可观测性**：接入 Langfuse / LangSmith 实现 LLM 调用全链路 Tracing
6. **安全防护**：输入过滤、输出审核、终端命令 Docker 沙箱隔离
7. **A2A 协议**：支持 Google A2A (Agent-to-Agent) 标准协议，实现跨平台 Agent 互操作
8. **云部署**：Docker Compose 一键部署，提供公网可访问 URL
9. **垂直领域扩展**：基于 PropertyApp 模式，快速复制到金融、医疗、法律等垂直场景

6.3 AI 能力演进路径



6.4 双模式扩展路径

当前	PropertyApp（房产）	XiaManus（通用）
近期	新增金融/医疗垂直 Agent	增加浏览器操作工具（Playwright）
中期	多垂直领域 Agent 自动路由	LangGraph 多 Agent 编排
远期	垂直 Agent 市场（插件化）	自主代码生成 + 自我进化

七、课程总结

7.1 个人收获

通过本项目，我从零构建了一个包含**双模式架构**的完整 AI Agent 系统，核心收获：

- **Agent 架构理解**：深入理解 ReAct 循环模式，掌握了 Agent 自主推理、工具选择、状态管理的完整流程
- **SDD 方法论**：践行了 Product Spec → Architecture Spec → API Spec 的规格驱动开发流程，先设计后编码
- **工具生态**：掌握了 Function Calling、MCP 协议、RAG 检索增强等前沿 AI 技术，理解了工具即能力的理念
- **双模式架构设计**：理解了"垂直深度 + 通用广度"的设计哲学——PropertyApp 深耕领域，XiaManus 覆盖通用场景
- **工程实践**：从硬编码 API Key 到环境变量管理，理解了安全开发的重要性
- **全栈能力**：Spring Boot 3.4 后端 + Vue 3 前端 + Docker 容器化部署

7.2 工程思维转变

- **从"调 API"到"编排 Agent"**: 不再仅仅调用 LLM API, 而是设计 Agent 的思考-行动循环和工具编排
 - **从"单一模式"到"双模式协同"**: 认识到专业领域和通用任务需要不同的 Agent 设计范式
 - **从"硬编码"到"配置驱动"**: API Key 通过环境变量注入, 遵循安全最佳实践
 - **从"代码优先"到"规格优先"**: 先明确 Product Spec, 再动手开发, 减少返工
 - **从"独立工具"到"协议集成"**: 通过 MCP 标准协议集成外部服务, 实现工具生态的即插即用
-

附录：仓库目录结构

本仓库严格遵循课程要求（第四节）的目录结构规范：

cs599-project/	← GitHub 仓库根目录
├── docs/	← 项目文档
│ └── CS599_大作业报告.md	← 本报告
├── src/	
│ └── xia-ai-agent-new/	← 主项目（Maven 多模块）
│ ├── docs/	← 报告副本（与根 docs/ 同步）
│ └── CS599_大作业报告.md	
│ ├── src/main/java/	← 后端 Java 源码
│ └── com/xiapi/xiaaiagent/	
│ ├── agent/	← Agent 核心（4 层继承体系）
│ ├── app/PropertyApp.java	← 房产分析助手
│ ├── tools/	← 7 个内置工具
│ ├── rag/	← RAG 检索增强
│ ├── chatmemory/	← 对话记忆
│ ├── advisor/	← Advisor 组件
│ └── controller/	← REST API
│ ├── src/main/resources/	← 配置文件
│ ├── src/test/	← 测试代码
│ ├── xia-ai-agent-frontend/	← Vue 3 前端
│ ├── xia-image-search-mcp-server/	← 自建 MCP Server
│ ├── Dockerfile	← 容器化
│ ├── pom.xml	← Maven 配置
│ └── README.md	← 项目说明
├── README.md	← 仓库总览 README
├── LICENSE	← MIT 开源协议
└── .gitignore	← Git 忽略规则

参考资料

- yu-ai-agent 开源项目（项目架构与实现参考）：<https://github.com/liyupi/yu-ai-agent>
- Spring AI 官方文档：<https://docs.spring.io/spring-ai/reference/>
- Spring AI Alibaba 文档：<https://sca.aliyun.com/>
- MCP 协议规范：<https://modelcontextprotocol.io/>
- OpenManus 开源项目：<https://github.com/mannaandpoem/OpenManus>

- ReAct 论文: Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models" , 2022
- PgVector 文档: <https://github.com/pgvector/pgvector>